

ADAL Backend — Latency Analysis Report

Overview

After reading through every service, route, and core module, there are **6 distinct categories** of latency issues ranging from critical blockers to minor inefficiencies. They are ordered from highest to lowest impact.

Critical Issues

1. InLegalBERT Embeddings Computed Serially, One Sentence at a Time

File: `[claim_service.py]`(file:///c:/Hasan/Desktop/Adal-FYP-cleanup-core/adal-backend/app/services/claim_service.py#L319-L329)

The `segment_claims_boundary_detection()` function calls `get_embeddings(sent)` for **every single sentence** in a document in a serial `for` loop. Each call goes through the BERT encoder individually:

```
# Line 321 – SERIAL, one sentence at a time
for sent in sentences:
    emb = get_embeddings(sent)  # ← full forward pass per sentence
    sentence_embeddings.append(emb)
```

Why it's slow:

- The model is loading correctly (global cache via `_model`), but batching is ignored completely.
- When citation-guided mode is active (line 747), `segment_claims_boundary_detection` runs on text **segments** around citation groups — not the full document. This reduces sentence counts somewhat, but the serial-per-sentence BERT call is still the bottleneck.
- In the **fallback path** (line 761, when no citations exist), it runs on the **entire document text** — a 20-page document can have 200+ sentences, meaning 200+ separate BERT forward passes.
- InLegalBERT is a 110M-parameter transformer. Each inference on CPU takes ~100–500 ms.
- Even in citation-guided mode, a document with 10 citation groups of ~20 sentences each = 200 serial calls. Total: **20–100 seconds** of pure embedding time.

Fix: Batch the sentences in one call:

```
# Replace the loop with a batched call
sentence_embeddings = []
batch_size = 16
for i in range(0, len(sentences), batch_size):
    batch = sentences[i:i+batch_size]
    inputs = tokenizer(batch, max_length=512, padding=True, truncation=True,
return_tensors="pt")
    inputs = {k: v.to(device) for k, v in inputs.items()}
    with torch.no_grad():
        out = model(**inputs)
```

```
embs = out.last_hidden_state[:, 0, :].cpu().numpy()
sentence_embeddings.extend(embs)
```

2. `find_sentence_boundaries_around_position` Tokenizes the Full Document Twice Per Citation Group

File: `[claim_service.py]`(file:///c:/Hasan/Desktop/Adal-FYP-cleanup-core/adal-backend/app/services/claim_service.py#L720-L737)

Inside `segment_claims_with_citations()`, for every citation group:

```
# Called TWICE per group – once for start, once for end
start, _ = find_sentence_boundaries_around_position(document.ocr_text,
first_citation_pos, ...)
_, end = find_sentence_boundaries_around_position(document.ocr_text,
last_citation_pos, ...)
```

And `find_sentence_boundaries_around_position` (line 164) calls `sent_tokenize(document_text)` on the **full document** every time it is called. For a 50-citation document this means `sent_tokenize` runs 100 times on the same text.

Fix: Tokenize sentences once before the loop and pass the result in:

```
sentences = sent_tokenize(document.ocr_text)
sentence_positions = _precompute_sentence_positions(document.ocr_text, sentences)

for group in citation_groups:
    start = _boundary_from_precomputed(sentence_positions, first_citation_pos,
...)
    end = _boundary_from_precomputed(sentence_positions, last_citation_pos, ...)
```

3. Drafting Agent Creates a New `psycopg2` Connection Per Request

File: `[drafting_service.py]`(file:///c:/Hasan/Desktop/Adal-FYP-cleanup-core/adal-backend/app/services/drafting_service.py#L131)

```
conn = psycopg2.connect(self.db_url, connect_timeout=5)
```

This raw `psycopg2.connect()` is called inside `retriever_node()` **on every chat message** to the drafting agent. Opening a new TCP connection to PostgreSQL takes ~50–200 ms and bypasses the existing SQLAlchemy connection pool entirely.

Fix: Use the SQLAlchemy engine that is already passed in via `db` (the `Session` parameter in `process_document`), or create a shared engine with a connection pool:

```
# At class __init__ or module level:
_engine = create_engine(db_url, pool_size=5, pool_pre_ping=True)

# In retriever_node, use:
with _engine.connect() as conn:
    rows = conn.execute(text("SELECT ..."), ...).fetchall()
```

● High-Impact Issues

4. Chat Retriever: Undersized Connection Pool + Per-Query Connection Checkout

File: `[chat_service.py](file:///c:/Hasan/Desktop/Adal-FYP-cleanup-core/adal-backend/app/services/chat_service.py#L86-L94)`

```
def _get_vector_engine():
    global _vector_engine
    if _vector_engine is None:
        url = os.getenv("LOCAL_DATABASE_URL")
        _vector_engine = create_engine(url, pool_size=3, pool_pre_ping=True)
    return _vector_engine
```

The global cache is correct — the engine is only created once. However, `pool_size=3` is very small. Under concurrent requests (or even a single user sending messages quickly), the retriever node will queue waiting for a free connection.

Additionally, the retriever runs **up to 3 separate SQL queries** (one per search query) each checking out a new connection from the pool:

```
for query_text in queries:
    with engine.connect() as conn:    # ← new connection per query
        rows = conn.execute(sql, ...).fetchall()
```

Fix: Reuse one connection across the loop:

```
with engine.connect() as conn:
    for query_text in queries:
        rows = conn.execute(sql, {"qvec": vec_literal}).fetchall()
    ...
```

And increase `pool_size` to at least 10.

5. OCR File Upload Is Fully Synchronous and Blocks the Event Loop

File: [file_service.py](file:///c:/Hasan/Desktop/Adal-FYP-cleanup-core/adal-backend/app/services/file_service.py#L562-L578)

```
# Step 5: Automatically extract text from the uploaded file (best effort)
try:
    text, _ = _extract_text_from_file(final_file_path) # ← BLOCKS entire server
    ...
```

`_extract_text_from_file` runs Tesseract OCR and pdf2image conversion synchronously inside an `async` route. For a 50-page PDF at DPI 300, this can block the process for **several minutes**, freezing all other concurrent requests.

Fix: Run OCR in a thread pool executor so the event loop stays free:

```
import asyncio
from concurrent.futures import ProcessPoolExecutor

loop = asyncio.get_event_loop()
text, method = await loop.run_in_executor(
    None, # or a ProcessPoolExecutor
    _extract_text_from_file,
    final_file_path
)
```

Or better: return immediately after saving the file and process OCR as a background task using FastAPI's `BackgroundTasks`.

6. Evidence Service Re-reads Full Files From Disk Per Result

File: [evidence_service.py](file:///c:/Hasan/Desktop/Adal-FYP-cleanup-core/adal-backend/app/services/evidence_service.py#L62-L69)

```
for result in retrieval_results:
    ...
    if filename and DATASET_DIR.exists():
        file_path = DATASET_DIR / filename
        if file_path.exists():
            with open(file_path, 'r', encoding='utf-8', errors='ignore') as f:
                full_text = f.read() # ← reads entire judgment file per result
```

For each of the `k=10` search results, the service opens and reads the full source document from disk. Supreme Court judgment text files can be hundreds of KB to MB in size. Reading 10 of them per verification call adds

significant I/O latency.

Fix: Cache recently-read files in a simple LRU cache:

```
from functools import lru_cache

@lru_cache(maxsize=128)
def _read_judgment_file(filepath: str) -> str:
    with open(filepath, 'r', encoding='utf-8', errors='ignore') as f:
        return f.read()
```

● Moderate Issues

7. `list_files` Makes a Filesystem `stat()` Call Per Document Row

File: `[file_service.py](file:///c:/Hasan/Desktop/Adal-FYP-cleanup-core/adal-backend/app/services/file_service.py#L647-L651)`

```
for doc in documents:
    file_size = _get_file_size(doc.filename)  # ← stat() syscall per document
    file_type = _infer_file_type(doc.filename)
```

`_get_file_size` calls `path.stat().st_size` — a filesystem syscall — for every document in the result page. For a list of 100 documents this is 100 syscalls.

Fix: Store `file_size` and `file_type` in the database at upload time (columns already referenced in code comments) and eliminate the per-row filesystem access.

8. Redis `is_connected()` Calls `ping()` on Every Cache Operation

File: `[redis_client.py](file:///c:/Hasan/Desktop/Adal-FYP-cleanup-core/adal-backend/app/core/redis_client.py#L96-L104)`

```
def is_connected(self) -> bool:
    if self._client is None:
        return False
    try:
        self._client.ping()  # ← round-trip to Redis on every cache_get/cache_set
        return True
```

Every call to `cache_get()` and `cache_set()` first calls `is_connected()` which does a `PING` to Redis. If Redis is local, this is ~0.1–0.5 ms — but it's wasted when Redis is already healthy and doubles the number of Redis round-trips.

Fix: Use a sticky boolean that gets reset only on connection error, or use `pool_pre_ping=True` (which already does health checks at the connection level):

```
def is_connected(self) -> bool:
    return self._client is not None # Trust the pool; only reset on exception
```

9. LLM Verification Has a 180-Second Timeout to a Local Ollama Instance

File: [llm_service.py](file:///c:/Hasan/Desktop/Adal-FYP-cleanup-core/adal-backend/app/services/llm_service.py#L150)

```
response = requests.post(
    f"{LLM_BASE_URL}/api/generate",
    json={...},
    timeout=180 # 3-minute timeout
)
```

The 3-minute timeout means a user request to verify a claim will block the server thread for up to 3 minutes if Ollama is slow or unavailable. There is no async handling and no streaming.

Fix: Either reduce the timeout to a reasonable value (30s) and return a proper error to the UI, or move the Ollama call into a background task and poll for the result.

Minor Issues

10. The `save_citations` Function Issues a `refresh()` Per Citation After Commit

File: [citation_service.py](file:///c:/Hasan/Desktop/Adal-FYP-cleanup-core/adal-backend/app/services/citation_service.py#L460-L463)

```
local_db.commit()
for db_citation in db_citations:
    local_db.refresh(db_citation) # ← N round-trips to DB
```

Each `refresh()` call emits a `SELECT` to reload the object. For 50+ citations this is 50 extra DB queries. Unless the post-commit values are actually needed, these can be removed or deferred.

11. Two Separate Count + Fetch Queries in `list_files`

File: [file_service.py](file:///c:/Hasan/Desktop/Adal-FYP-cleanup-core/adal-backend/app/services/file_service.py#L637-L644)

```
total_count = local_db.query(Document).count()    # ← query 1
documents = local_db.query(Document)...all()      # ← query 2
```

This can be done with a single query using `SELECT COUNT(*) OVER() ... FROM documents` or by using a `func.count` window function, eliminating one round-trip.

Summary Table

#	Location	Issue	Severity	Estimated Latency Impact
1	<code>claim_service.py</code>	BERT embeddings computed serially sentence-by-sentence	● Critical	+20–100 sec per document
2	<code>claim_service.py</code>	<code>sent_tokenize</code> called on full doc for every citation	● Critical	+5–30 sec per document
3	<code>drafting_service.py</code>	New raw psycopg2 connection per chat message	● Critical	+50–200 ms per chat
4	<code>chat_service.py</code>	New DB connection per search query + tiny pool	● High	+100–500 ms per chat
5	<code>file_service.py</code>	OCR blocks async event loop	● High	Freezes server for minutes
6	<code>evidence_service.py</code>	Full judgment files read from disk per result	● High	+100 ms–2 sec per retrieval
7	<code>file_service.py</code>	<code>stat()</code> syscall per row in <code>list_files</code>	● Moderate	+10–50 ms per listing
8	<code>redis_client.py</code>	<code>ping()</code> called on every cache access	● Moderate	+0.1–0.5 ms per cache call
9	<code>llm_service.py</code>	3-min blocking timeout to Ollama	● Moderate	Up to 3 min blocked
10	<code>citation_service.py</code>	N <code>refresh()</code> calls after bulk insert	● Minor	+5–20 ms per extraction
11	<code>file_service.py</code>	Two queries instead of one for paginated list	● Minor	+5–10 ms per list call

[!IMPORTANT] **Issues #1 and #2 together** are almost certainly the root cause of the worst latency you're experiencing, since claim segmentation runs BERT inference sentence-by-sentence across the full document. Fixing batching alone could cut claim processing time by 10–50×.